

High-Level Design ? Assignment 2: Drone Mapper Simulation

Architecture Overview

Assignment 2 refactors the drone mapping system into a fully decoupled, testable component hierarchy.

All components communicate through well-defined interfaces, with dependency injection enabling independent testing of each component.

Component Hierarchy

```
main()
  ??? SimulationManager
    ??? ISimulationRunFactory
      ??? SimulationRunImpl (one per combination)
        ??? IMissionControl (MissionControlImpl)
        ?      ??? IDroneControl (DroneControlImpl)
        ?      ??? IMappingAlgorithm (MappingAlgorithmImpl)
        ?      ??? ILidar (MockLidar)
        ?      ??? IGPS (MockGPS)
        ?      ??? IDroneMovement (MockMovement)
        ??? IMap3D (hidden, Map3DImpl from .npy)
        ??? IMutableMap3D (output, Map3DImpl empty)
```

Class Diagram

```
classDiagram
    direction TB

    class ISimulation {
        <<interface>>
        +run(composition, output_path) report
    }
    class ISimulationRun {
        <<interface>>
        +run() SimulationResult
    }
    class ISimulationRunFactory {
        <<interface>>
        +create(sim,mission,drone,lidar,output_path) ISimulationRun
    }
    class IMissionControl {
        <<interface>>
        +runMission() MissionRunResult
    }
    class IDroneControl {
        <<interface>>
        +step() DroneStepResult
        +state() DroneState
    }
    class ILidar {
```

```

    <<interface>>
    +scan(orientation) LidarScanResult
    +config() LidarConfigData
}
class IGPS {
    <<interface>>
    +position() Position3D
    +heading() Orientation
}
class IDroneMovement {
    <<interface>>
    +rotate(dir, angle) MovementResult
    +advance(distance) MovementResult
    +elevate(distance) MovementResult
}
class IMappingAlgorithm {
    <<interface>>
    +nextStep(state, scan*) MappingStepCommand
}
class IMap3D {
    <<interface>>
    +atVoxel(pos) VoxelOccupancy
    +getMapConfig() MapConfig
    +isInBounds(pos) bool
}
class IMutableMap3D {
    <<interface>>
    +set(pos, value) void
    +save(path) void
}

class SimulationManager {
    -unique_ptr~ISimulationRunFactory~ run_factory_
}
class SimulationRunFactoryImpl {
    +create(...) ISimulationRun
}
class SimulationRunImpl {
    -unique_ptr~IMutableMap3D~ hidden_map_
    -unique_ptr~IMutableMap3D~ output_map_
    -unique_ptr~IGPS~ gps_
    -...
}
class MissionControlImpl {
    -MissionConfigData mission_
    -IDroneControl& drone_control_
    -IMutableMap3D& output_map_
}
class DroneControlImpl {
    -IMappingAlgorithm& algorithm_
    -ILidar& lidar_
    -IGPS& gps_
    -IDroneMovement& movement_
}

```

```

        -ImmutableMap3D& output_map_
    }
    class MappingAlgorithmImpl {
        -BFS frontier
        -scan queue
        -nav path
    }
    class MockLidar {
        -LidarConfigData config_
        -IMap3D& map_
        -IGPS& gps_
    }
    class MockGPS {
        -Position3D position_
        -Orientation heading_
    }
    class MockMovement {
        -MockGPS& gps_
        -IMap3D& map_
        -PhysicalLength drone_radius_
    }
    class Map3DImpl {
        -shared_ptr~NpyArray~ map_
        -MapConfig config_
    }
    class MapsComparison {
        +compare(origin, targets) vector~double~
    }
    class ScanResultToVoxels {
        +applyToMap(output, origin, heading, scan, lidar) void
    }
    class NpyArray {
        -vector~uint8_t~ data_
        +load(path) NpyArray
        +save(path) void
        +at(x,y,z) bool
        +set(x,y,z,val) void
    }
}

```

```

ISimulation <|.. SimulationManager
ISimulationRunFactory <|.. SimulationRunFactoryImpl
ISimulationRun <|.. SimulationRunImpl
IMissionControl <|.. MissionControlImpl
IDroneControl <|.. DroneControlImpl
ILidar <|.. MockLidar
IGPS <|.. MockGPS
IDroneMovement <|.. MockMovement
IMappingAlgorithm <|.. MappingAlgorithmImpl
IMap3D <|-- ImmutableMap3D
ImmutableMap3D <|.. Map3DImpl
Map3DImpl --> NpyArray

```

```

SimulationManager --> ISimulationRunFactory

```

```

SimulationRunImpl --> IMutableMap3D
SimulationRunImpl --> IGPS
SimulationRunImpl --> IDroneMovement
SimulationRunImpl --> ILidar
SimulationRunImpl --> IMappingAlgorithm
SimulationRunImpl --> IDroneControl
SimulationRunImpl --> IMissionControl
MissionControlImpl --> IDroneControl
MissionControlImpl --> IMutableMap3D
DroneControlImpl --> IMappingAlgorithm
DroneControlImpl --> ILidar
DroneControlImpl --> IGPS
DroneControlImpl --> IDroneMovement
DroneControlImpl --> IMutableMap3D
MockLidar --> IMap3D
MockMovement --> MockGPS
MockMovement --> IMap3D

```

Sequence Diagram ? Simulation Run

sequenceDiagram

```

participant Main
participant Manager as SimulationManager
participant Factory as SimulationRunFactoryImpl
participant Run as SimulationRunImpl
participant Mission as MissionControlImpl
participant Drone as DroneControlImpl
participant Algo as MappingAlgorithmImpl
participant Lidar as MockLidar
participant Movement as MockMovement
participant Map as IMutableMap3D

```

```

Main->>Manager: run(composition, output_path)
loop each (simulation × mission × drone × lidar)
    Manager->>Factory: create(sim, mission, drone, lidar, output_path)
    Factory->>Factory: load hidden_map from .npz
    Factory->>Factory: create output_map (empty NpyArray)
    Factory-->>Manager: SimulationRunImpl
    Manager->>Run: run()
    Run->>Mission: runMission()
    loop each step (up to max_steps)
        Mission->>Drone: step()
        Drone->>Algo: nextStep(state, last_scan*)
        Algo-->>Drone: MappingStepCommand
        opt has movement
            Drone->>Movement: rotate/advance/elevate
            Movement-->>Drone: MovementResult
        end
        opt has scan_orientation
            Drone->>Lidar: scan(orientation)
            Lidar-->>Drone: LidarScanResult
            Drone->>Map: ScanResultToVoxels::applyToMap(...)
        end
    end
end

```

```

        end
        Drone-->>Mission: DroneStepResult
    end
    Mission->>Map: save(output_path)
    Mission-->>Run: MissionRunResult
    Run->>Run: MapsComparison::compare(hidden, output)
    Run-->>Manager: SimulationResult
end
Manager-->>Main: SimulationManagerReport

```

Design Decisions

1. Self-Contained NPY I/O (No TinyNPY Dependency)

We implement our own minimal NpyArray class that reads/writes the .npz format directly. This eliminates an external dependency while remaining fully compatible with NumPy's output.

2. MockMovement Includes Collision Detection

Unlike the skeleton stub, MockMovement::advance() and elevate() check the hidden map for obstacles along the path. This prevents the drone from flying through walls, which would make the simulation physically invalid.

3. BFS Exploration Algorithm

MappingAlgorithmImpl uses a breadth-first exploration strategy:

1. Initial scan: at the starting position, emit 10 scan directions (8 horizontal + up/down)
2. Queue neighbors: after each scan, queue all unmapped adjacent voxels
3. Navigate: BFS pathfinding to the next frontier voxel
4. Rescan: on arrival at a new position, scan again
5. Terminate: when the frontier is exhausted (no reachable unmapped voxels)

4. ScanResultToVoxels (Provided Algorithm)

The ScanResultToVoxels::applyToMap() method (adapted from the course skeleton) converts LiDAR hits into map occupancy states. It marks:

- Path before a hit as Empty
- Hit position as Occupied
- Near-field returns (<z_min) as PotentiallyOccupied

5. Map Comparison Algorithm

MapsComparison::compare() computes a score 0..100 based on:

- Voxel-wise agreement between the hidden map (reference) and the output map
- False positives (output says occupied, reference says empty) are penalized
- Score = correct / (total + false_positives) * 100

6. Error Handling

- ErrorHandler (singleton) immediately flushes all errors to error_log.txt
- Simulation continues after per-run errors (score = -1 for failed runs)
- Fatal configuration errors abort with a message to stderr

File Structure

```

HW2_CPP/
??? CMakeLists.txt
??? README.md
??? HLD.md
??? include/drone_mapper/      ? All public headers (interfaces + implementations)

```

? ??? types/	? Data type headers
? ??? Units.h	? mp-units type aliases
? ??? I*.h	? Interface headers
? ??? *Impl.h	? Implementation headers
? ??? Mock*.h	? Mock component headers
? ??? ...	
??? src/	? Implementation (.cpp) + private helpers
? ??? *Impl.cpp	
? ??? Mock*.cpp	
? ??? YamlParser.{h,cpp}	
? ??? ErrorHandler.{h,cpp}	
? ??? drone_mapper_simulation_main.cpp	
? ??? maps_comparison_main.cpp	
??? tests/	
? ??? components/	? 7 component test files
? ??? integration/	? 2 integration test files
??? inputs/	
??? drone/	? Drone YAML configs
??? lidar/	? Lidar YAML configs
??? mission/	? Mission YAML configs
??? simulation/	? Simulation YAML configs
??? map/	? .npy map files + generator
??? sim_compose.yaml	? Simulation composition file